NEXORA: A SMART EVENT MANAGEMENT SYSTEM FOR COLLEGES

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

CHILUKURI VENNELA REDDY (VTU25619)

10211CS224
Full Stack Application Development
Slot : E2-B20

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”Nexora: A Smart Event Management System for Colleges” by Chilukuri Vennela Reddy (VTU25619) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hemamalini. S

M.E., Ph.D.

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr.Sumithra M.

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Chilukuri Vennela Reddy)

Date:06/05/2026

ABSTRACT

This project presents a web-based College Event Management System designed to simplify event organization, student registration, and certification processes. The system provides secure authentication with email verification and JWT[3]-based login for students and administrators. Students can browse available events, register, and complete payments, while administrators manage event details and track attendance by marking students as present or absent. Based on attendance, the system automatically generates certificates in PDF format, which students can download or receive via email, ensuring faster and more efficient certification. The application is built using React.js for the frontend and PHP with MySQL for the backend, with additional integrations such as SendGrid[1] for email services and FPDF for PDF generation. Overall, the system reduces manual work, improves efficiency, and offers a scalable solution for managing college events.

Keywords: Event Management System, Student Registration, JWT Authentication, Attendance Tracking, PDF Certificate Generation, React.js

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
JWT	JSON Web Token
PDF	Portable Document Format
DBMS	Database Management System
SQL	Structured Query Language
UI	User Interface
UX	User Experience
HTTP	HyperText Transfer Protocol
SMTP	Simple Mail Transfer Protocol
CRUD	Create, Read, Update, Delete

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

The rapid growth of digital technologies has transformed the way academic institutions manage their activities, including the organization of events. In many colleges, event management is still handled manually or through fragmented systems, leading to inefficiencies, miscommunication, and delays in registration, attendance tracking, and certification. To address these challenges, this project proposes a web-based[5] College Event Management System that provides a centralized platform for students and administrators. The system enables students to securely register, verify their email, explore events, and complete registrations, while administrators can create and manage events, monitor participation, and track attendance efficiently. By automating key processes such as attendance marking and certificate generation, the system reduces manual effort and ensures timely delivery of digi-

tal certificates. Overall, the proposed solution enhances efficiency, improves user experience, and supports seamless management of college events through a reliable and scalable web application.

1.2 Aim of the Project

The aim of this project is to develop a web-based College Event Management System that provides a centralized platform for managing academic events efficiently. The system aims to simplify student registration, ensure secure authentication through email verification and JWT, enable seamless event discovery and participation, and allow administrators to manage events and track attendance effectively. It also aims to automate certificate generation in PDF format and deliver them quickly to students, thereby reducing manual effort, improving accuracy, and enhancing overall user experience.

1.3 Scope of the Project

The scope of this project includes the design and development of a web-based College Event Management System that supports secure user authentication, event creation and management, student registration, and attendance tracking. The system enables students to browse and register for events, complete payments, and receive certificates

in digital PDF format based on their attendance status. Administrators can efficiently manage event details, monitor participation, and control access to system functionalities. The project is limited to web-based access using a React frontend and PHP-MySQL[3] backend, with integrations for email notifications and certificate generation. Future enhancements may include mobile application support, advanced analytics, AI-based event recommendations, and integration with external payment gateways and institutional systems.

1.4 Framework

The proposed system follows a client–server architecture using a modern web development framework. The frontend is developed using React.js, which provides a dynamic and responsive user interface for students and administrators. The backend is implemented using PHP, which handles business logic, authentication, and communication with the database. MySQL is used as the database management system to store user details, event information, registrations, and attendance records. The system uses RESTful APIs for communication between the frontend and backend, with JWT (JSON Web Tokens)[5] ensuring secure authentication. Additional integrations such as SendGrid are used for email verification and notifications, while FPDF is used for

generating digital certificates in PDF format. The entire system is deployed in a local server environment using XAMPP, ensuring smooth development and testing. This framework[6] ensures scalability, security, and efficient data management.

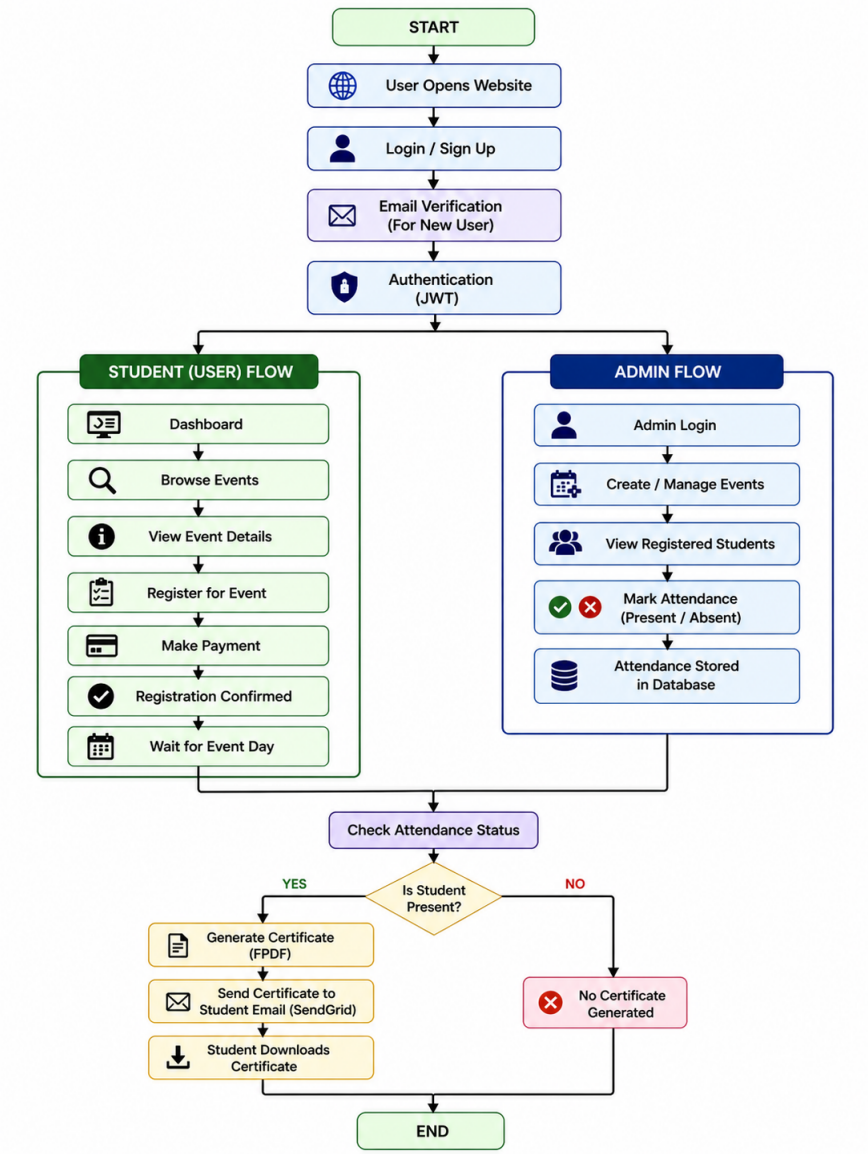

Figure 1.1: Workflow of Nexora Smart Event Management System

The Figure 1.1 illustrates the workflow of the Nexora Smart Event Management System. It begins with the user accessing the website, followed by login or signup and authentication. Based on the user role,

the system directs either to the student flow or admin flow. Students can browse events, register, make payments, and track their participation. Administrators can create and manage events, view registrations, and mark attendance. The system then verifies attendance status to generate certificates for eligible users, ensuring a complete and efficient event management process.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Many software applications are available in the market for managing events efficiently. These applications help organizations, companies, and educational institutions to organize events, handle registrations, manage participants, track attendance, and send notifications. Studying similar applications helps in understanding their features, advantages, and limitations. This analysis is useful for developing the Smart College Event Platform with improved features and better user experience. Some popular applications in the market include[7] Eventbrite[8], Cvent[9], Whova, and CampusGroups. Eventbrite[9] is widely used for creating events, selling tickets, managing registrations, and promoting events online. It is suitable for workshops, seminars, concerts, and conferences. Whova is known for mobile event applications, networking features, schedules, surveys, and communication

tools for attendees. CampusGroups is specially designed for colleges and universities, helping manage student clubs, campus activities, and event registrations. Although these applications provide many useful features, they also have certain limitations. Many systems are expensive for small colleges and institutions with limited budgets. Some applications are mainly designed for business or commercial events rather than educational purposes. In many cases, customization for department-level college events is limited. Attendance tracking and certificate generation may require additional tools or paid services. Some systems also have complex interfaces that may be difficult for first-time users. The Nexora: A Smart Event Management System for Colleges is developed to overcome these limitations by providing a simple, affordable, and college-oriented solution. It combines all essential features such as event creation, registration management, attendance tracking, notifications, and certificate generation in one system. This platform is easy to use and specifically designed to meet the requirements of students, faculty members, and administrators. After analyzing similar applications available in the market, it is clear that existing systems are useful but not fully suitable for educational institutions. Therefore, the Nexora Platform offers a better and customized solution for colleges and universities.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

The frontend of the Smart College Event Platform is developed using React.js with Vite as the build tool. It provides a fast, responsive, and interactive user interface for students, organizers, and administrators. The frontend handles user interactions such as registration, login, event browsing, and certificate access, ensuring a smooth and user-friendly experience. **3.1.1 Environment Setup**

The development environment requires the following tools:

- XAMPP Server
- Apache and MySQL services
- PHP support
- phpMyAdmin for database management

The setup procedure is as follows:

1. Install XAMPP.
2. Start Apache and MySQL services.
3. Place backend files inside the *htdocs* folder.
4. Configure the database connection in the PHP files.

3.1.2 Structure of the Project The backend folder structure consists of the following components:

- **api/** – Contains API endpoints
- **config/** – Database configuration files
- **auth/** – Handles login and JWT authentication
- **events/** – Manages event creation and updates
- **certificates/** – Generates PDF certificates
- **notifications/** – Handles email services

3.1.3 Execution Procedure

1. Start Apache and MySQL services in XAMPP.
2. Import the database using phpMyAdmin.
3. Run the backend using localhost.
4. Connect the frontend application with backend APIs.

3.2 Backend Implementation

The backend of the Smart College Event Platform is developed using PHP and MySQL. It is responsible for handling business logic, processing user requests, managing authentication, storing data, and connecting the frontend with the database. The backend ensures secure communication between users and the system while maintaining smooth performance

3.2.1 Environment Setup

To set up the backend environment, XAMPP[8] server is used because it provides Apache, PHP, and MySQL in one package. After installing XAMPP, Apache and MySQL services must be started from the control panel. The project backend files are placed inside the htdocs folder. A code editor such as Visual Studio Code can be used for writing and editing PHP files. phpMyAdmin is used for database creation and management

3.2.2 Structure of the Project

The backend project is organized into multiple folders and files for better maintenance and development. Configuration files are used for database connection settings. API files handle requests from the frontend. Authentication files manage login, registration, and JWT token validation. Event management files perform create, update, delete, and

view operations. Notification files handle email sending, while certificate files generate PDF certificates. This modular structure improves readability and scalability of the application.

3.2.3 Execution Procedure

To execute the backend, first start Apache and MySQL[8] services in XAMPP. Then open phpMyAdmin and import the project database file. After that, place the backend project folder inside htdocs. Open a browser and run the project using localhost. The frontend communicates with backend APIs through HTTP requests. Once the connection is established, users can log in, manage events, and perform other operations successfully.

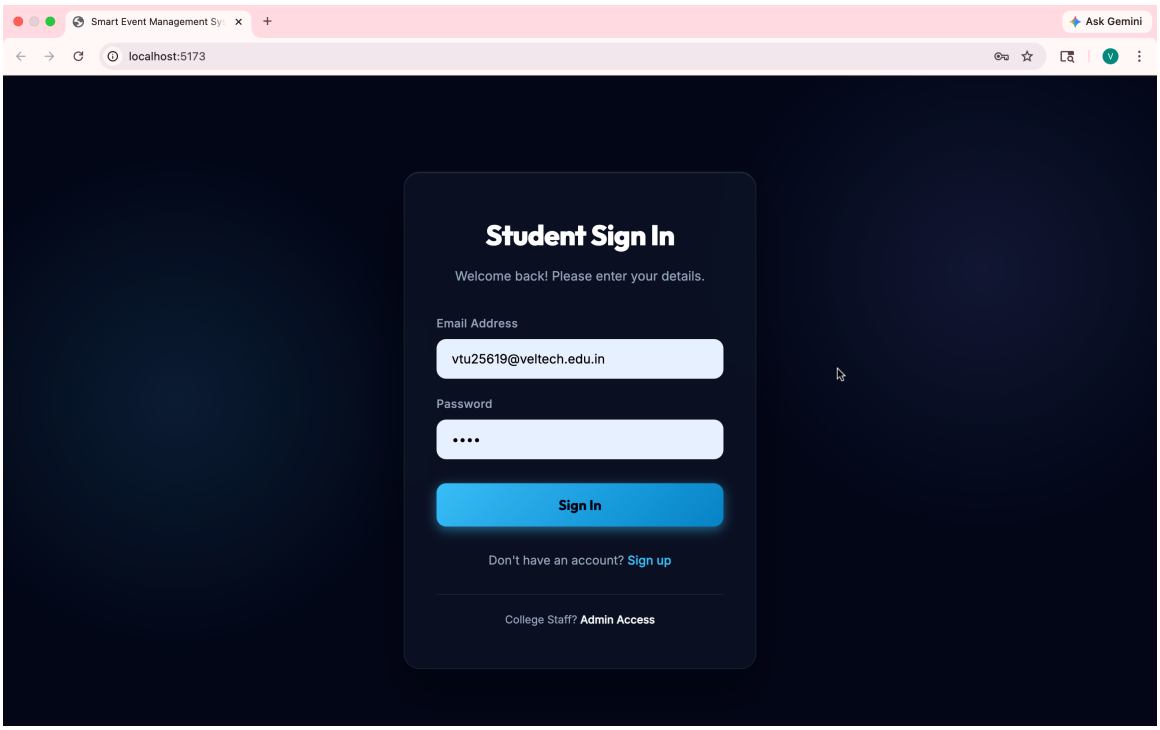

Figure 3.1: Login page of Nexora system

The figure 3.1 Shows the login page of the Nexora system, where

users enter their credentials to securely access the platform. After successful authentication, users are directed to their respective dashboards for further operations.

3.3 Database Implementation

The database of the Smart College Event Platform is developed using MySQL. It stores all important information such as user details, event data, registrations, attendance records, and certificates. The database is designed to provide fast access, secure storage, and proper relationship management between tables.

3.3.1 Database Configuration

Database configuration is done by creating a database in phpMyAdmin and connecting it through PHP using PDO. The host name is localhost, username is root, and password is left empty in the default XAMPP[9] setup. The database name is configured in the connection file to establish communication between the application and MySQL server.

3.1.2 Schema Structure

The schema structure is designed using relational database concepts. Different tables are connected using primary keys and foreign keys. The users table stores login and profile details. The events table stores event information. Registration[8] and attendance tables are linked

with users and events. Certificate records are stored separately for future reference. This schema avoids data redundancy and improves efficiency.

3.1.3 Tables created

Several tables are created for the proper functioning of the system. The users table stores student and admin information. The events table stores event title, description, date, and venue. The registrations table stores student participation details. The attendance table stores check-in records. The certificates table stores generated certificate information. Notification logs may also be stored for tracking sent [8]messages. These tables together support all operations of the platform.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Smart College Event Platform follows a three-tier architecture consisting of frontend, backend, and database layers. The frontend is developed using React.js, which provides an interactive user interface for students, organizers, and administrators. The backend is developed using PHP, which handles business logic, authentication, and API communication. The database layer uses MySQL[8] for storing user information, event details, registrations, attendance records, and certificates. This architecture provides better performance, scalability, and easy maintenance.

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) for the Smart College Event Management System illustrates how information flows from the user interface

to the backend and persistent storage.

Level 0: Represents the basic interaction where the User (Student/Administrator) inputs data such as login credentials, event[8] registration details, and attendance updates, and receives output in the form of event information, registration confirmation, and certificates.

Level 1: Details the processes of Authentication, Event Management, Registration Handling, Attendance Tracking, and Certificate Generation. Data flows from the React-based user interface through API requests to the PHP backend, where business logic is processed and data is stored in MySQL tables. The system also interacts with external services, where email notifications are handled via SendGrid[9] and certificates are generated using FPDF, with the generated data and records stored back in the database.

4.3 Module 1: Authentication Access Control Module

This module is responsible for handling user authentication, secure login, and access control within the system. It ensures that only authorized users can access protected functionalities.

- **User Registration:** The system allows users such as students, organizers[9], and administrators to register. Input validation is performed to ensure correctness of user data. Passwords are securely

stored using hashing techniques such as bcrypt to prevent unauthorized access.

- **User Login:** Users provide credentials (username/email and password), which are verified with stored data. Proper error handling is implemented for invalid login attempts.
- **JWT-Based Authentication:** After successful login, a JSON Web Token (JWT)[7] is generated. This token contains encoded user details such as user ID and role, and is used for maintaining session state securely.
- **Session Management:** The system follows a stateless session mechanism using JWT. The token is sent with each request and verified by the server without storing session data.
- **Role-Based Access Control (RBAC):** Different roles such as student, organizer, and administrator are assigned specific permissions:
  - Students can register and participate in events
  - Organizers can create and manage events
  - Administrators have full system control
- **Authorization Middleware:** Middleware functions are used to verify JWT tokens and check user roles before granting access to

specific resources or routes.

- **Security Features:** Token expiration is implemented to enhance security. Expired tokens require re-authentication. Additional features such as password reset and logout functionality can also be included.
- **Data Protection:** Sensitive user information is protected using encryption and secure communication protocols to prevent data breaches.

This module plays a crucial role in ensuring system security, protecting user data, and enforcing controlled access across all system functionalities.

4.4 Module 2: Event Management Registration Module

This module is responsible for creating, updating, managing, and organizing events within the system. It allows organizers and administrators to efficiently handle all event-related activities, while students can explore and participate in events.

- **Event Creation:** Organizers and administrators can create new events by providing details such as title, date, time, venue, description, and category. Proper validation ensures accurate data entry.

- **Event Updating:** Existing event details can be modified when required, such as changes in schedule, venue, or description. This ensures that users always have updated information.
- **Event Deletion:** Events that are cancelled or no longer required can be removed from the system by authorized users.
- **Event Viewing:** Students can view a list of available events along with detailed information including date, venue, and description. Events can be displayed in an organized manner for easy access.
- **Event Registration:** Students can register for events through the platform. The system records participant details and confirms successful registration.
- **Event Categorization:** Events can be classified into categories such as technical, cultural, or sports, making it easier for users to filter and find relevant events.
- **Participant Management:** Organizers can view and manage the list of registered participants for each event.
- **Notifications and Updates:** Users can be notified about new events or updates to existing events, ensuring better communication.
- **Data Management:** All event-related data is stored and managed efficiently in the database, ensuring quick retrieval and consis-

tency.

This module plays a vital role in simplifying the process of event organization and participation, making it easier for both organizers and students to interact with the system.

4.5 Module 3: Attendance Certification Module

This module handles participant attendance during the event and certificate generation after successful participation. Organizers can mark attendance manually or digitally. Once attendance is confirmed, participation certificates are generated automatically in PDF format and sent to students through email notifications.

Advantages of this Project: This project provides an efficient and user-friendly solution for managing college events. It reduces manual paperwork and saves time for administrators. Students can easily discover and register for events online. Attendance tracking becomes accurate and systematic. Automatic certificate generation improves convenience. Real-time notifications keep users updated about events and schedules.

a)Cost Effective: The system is affordable for colleges because it uses open-source technologies such as React.js, PHP, and MySQL.

b Easy to Use: The platform has a simple interface that allows stu-

dents and administrators to use it without difficulty.

c)Time Saving: Automation of registration, attendance, and certificate generation reduces manual effort and saves valuable time.

Disadvantages: The project depends on internet connectivity for proper functioning. Users without basic technical knowledge may require guidance during initial use. Regular maintenance is required for server performance and data security. If the server fails, the platform may become temporarily unavailable.

Chapter 5

RESULTS AND DISCUSSIONS

The proposed Smart College Event Management System was evaluated based on functionality, efficiency, and usability. The system was compared with the traditional manual method of managing college events. The results show significant improvements in automation, accuracy, and user experience.

Table 5.1: Comparision table for existing and traditional systems

Criteria	**Existing System**	**Proposed System**
Registration	Manual process	Online registration
Attendance	Manual marking	Digital tracking
Certificates	Manually issued	Auto PDF generation
Time	Time-consuming	Fast processing
Errors	High chances	Reduced errors

The Table 5.1 compares the existing manual system with the proposed automated system, highlighting improvements in efficiency, accuracy, and time through digital event management.

The figure 5.1 Shows the student dashboard of the Nexora system,

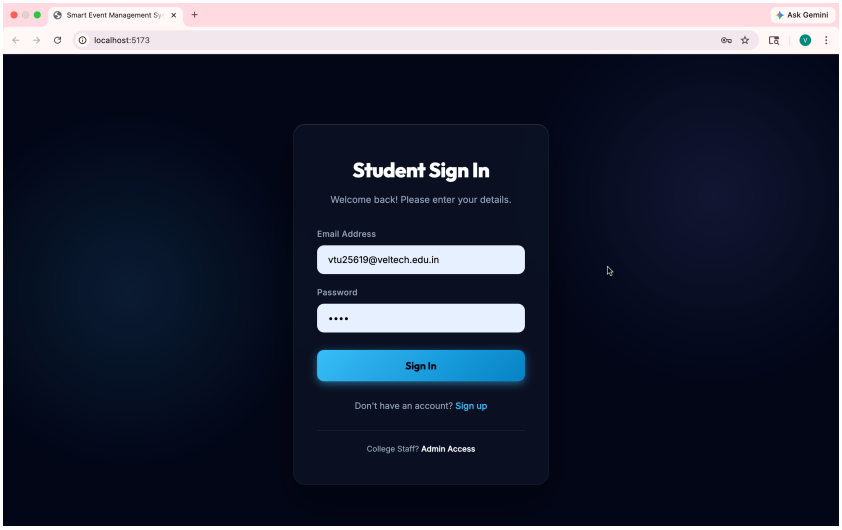

Figure 5.1: Student Dashboard

where users can view available events, register for them, and track their participation details. It provides an easy-to-use interface for managing event activities.

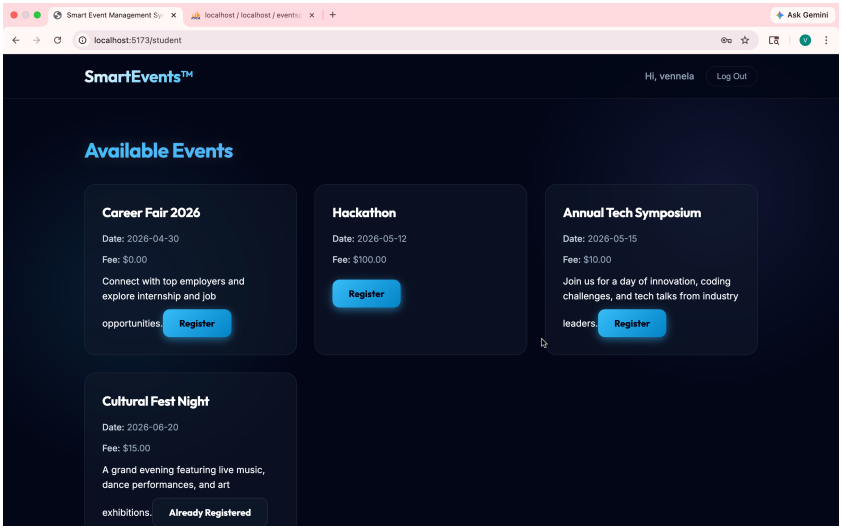

Figure 5.2: Admin Dashboard

The figure 5.2 the admin dashboard, which allows administrators to manage events, monitor registrations, and control system operations efficiently. It serves as the central interface for event administration

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart College Event Management System successfully provides a centralized and efficient platform for managing college events. It simplifies key processes such as student registration, event management, attendance tracking, and certificate[9] generation. By using secure authentication and automated workflows, the system reduces manual effort, minimizes errors, and improves overall efficiency. Students benefit from easy access to events and quick certification[8], while administrators can manage activities in a structured and organized manner. Overall, the system enhances user experience and offers a reliable solution for modernizing event management in academic institutions.

6.2 Future Enhancements

- Integration of secure online payment gateways
- Development of a mobile application
- QR code-based attendance tracking
- Advanced analytics and reporting dashboard
- Cloud deployment for better scalability and performance

Chapter 7
ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

Table 7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of knowledge	Requires in depth engineering knowledge	WK3,WK 4, WK5, WK6	Requires knowledge in Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies (Thymeleaf, HTML/CSS) for developing a scalable event management system.
WP2	Conflicting requirements	Technical and non -technical conflicts	WK 4, WK 6 WK 5,	Balances usability and security by ensuring easy student access while maintaining secure admin authentication and data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK 3, WK 5 WK 4,	Involves system design, database schema creation, validation handling, and efficient event filtering and registration mechanisms.
WP4	Familiarity	Partially new problems	WK4, WK8	Integrating real-time event updates, user interaction, and admin analytics introduces challenges beyond traditional manual systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for event filtering, registration tracking, and statistics generation not fully covered by standard templates.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK 5, WK 7 WK 6,	Addresses requirements of students, faculty coordinators, and administrators ensuring smooth event organization and participation.
WP7	Interdependence	System level integration	WK 3, WK 6 WK 5,	Integrates frontend UI, backend services, database, validation, and security modules into a unified system.

The project is analyzed based on Complex Engineering Problem (CEP) attributes such as depth of knowledge, conflicting requirements, and system integration. It requires strong technical knowledge in web development and database systems. The project also handles real-world challenges like security, usability, and multiple stakeholders. These factors confirm that the project qualifies as a complex engineering problem.

Table 7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system, extending beyond traditional event handling methods.

The project is mapped to Complex Engineering Activities (CEA) including use of diverse technologies and complex module interactions. It demonstrates innovation through a centralized web-based system. The project improves efficiency and user experience while reducing manual work. Hence, it satisfies the characteristics of complex engineering activities

Table 7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized event management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through authentication and controlled access.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management

The project is aligned with Sustainable Development Goals (SDGs) such as Quality Education and Industry Innovation. It supports digital transformation in educational institutions

References

[1] Meta Platforms Inc. (2024), “React: A JavaScript Library for Building User Interfaces,” React Official Documentation, 1(1), pp. 1–10.

[2] PHP Group (2024), “PHP: Hypertext Preprocessor Documentation for Backend Development,” PHP Official Documentation, 2(1), pp. 1–20.

[3] Oracle Corporation (2024), “MySQL Relational Database Management System for Data Storage,” MySQL Official Documentation, 3(2), pp. 1–15.

[4] OpenJS Foundation (2024), “Node.js Runtime Environment for Server-Side Applications,” Node.js Official Documentation, 2(1), pp. 1–12.

[5] Axios Contributors (2024), “Axios HTTP Client Library for API Communication,” GitHub Documentation, 2(1), pp. 5–12.

[6] JWT.io (2024), “JSON Web Token (JWT) Authentication and Authorization Mechanism,” Auth0 Documentation, 1(1), pp. 1–8.

[7] Eventbrite Inc. (2023), “Online Event Registration and Ticketing Platform Architecture,” Eventbrite Engineering Journal, 4(3), pp. 30–38.

[8] Cvent Inc. (2023), “Enterprise-Level Event Management System with Analytics and Reporting,” Cvent Technical Documentation, 5(2), pp. 15–25.

[9] Zoho Corporation (2023), “Zoho Backstage: Event Planning, Registration, and Engagement Platform,” Zoho Product Documentation, 3(1), pp. 10–18.

[10] Mozilla Foundation (2024), “JavaScript Programming and Web API Guide for Frontend Development,” MDN Web Docs, 6(2), pp. 1–30.

```
1
2  /* Database Schema & Core Backend API */
3  CREATE DATABASE IF NOT EXISTS smart_events_db;
4  USE smart_events_db;
5
6  CREATE TABLE users (
7    id INT AUTO_INCREMENT PRIMARY KEY,
8    name VARCHAR(255) NOT NULL,
9    email VARCHAR(255) NOT NULL UNIQUE,
10   password VARCHAR(255) NOT NULL,
11   role ENUM('student', 'admin') DEFAULT 'student',
12   is_verified BOOLEAN DEFAULT 0,
13   otp VARCHAR(6) NULL
14 );
15
16 CREATE TABLE events (
17   id INT AUTO_INCREMENT PRIMARY KEY,
18   title VARCHAR(255) NOT NULL,
19   description TEXT,
20   date DATE NOT NULL,
21   fee DECIMAL(10,2) DEFAULT 0.00
22 );
23
24 CREATE TABLE registrations (
25   id INT AUTO_INCREMENT PRIMARY KEY,
26   user_id INT NOT NULL,
27   event_id INT NOT NULL,
28   payment_status ENUM('paid', 'unpaid') DEFAULT 'unpaid',
29   attendance_status ENUM('present', 'absent', 'pending') DEFAULT 'pending',
30   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
31   FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE,
32   UNIQUE KEY (user_id, event_id)
33 );
34
35 /* db.php */
36 $pdo = new PDO("mysql:host=127.0.0.1;dbname=smart_events_db;charset=utf8mb4",
37 "root","",
38 [
39  PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
40  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
41 ]);
42
43 /* jwt_helper.php */
44 class JwtHelper {
45     private static $secret = 'secret123';
46
47     public static function encode($payload) {
48         $h = base64_encode(json_encode(['typ'=>'JWT','alg'=>'HS256']));
49         $p = base64_encode(json_encode($payload));
50         $s = base64_encode(hash_hmac('sha256',"$h.$p",self::$secret,true));
```

```
        return "$h.$p.$s";
    }

    public static function decode($jwt) {
        $t = explode('.',$jwt);
        return json_decode(base64_decode($t[1]),true);
    }
}

/* auth.php */
require_once 'init.php';

if($_SERVER['REQUEST_METHOD']==='POST'){
    $data = json_decode(file_get_contents("php://input"));

    $hash = password_hash($data->password, PASSWORD_BCRYPT);

    $pdo->prepare("INSERT INTO users (name,email,password)
    VALUES (?,?,?)")->execute([
        $data->name,
        $data->email,
        $hash
    ]);

    echo json_encode(["message"=>"User Created"]);
}

/* React Frontend */
import React, { useState } from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';

function App() {
  const [user, setUser] = useState(null);

  return (
    <Router>
      <Routes>
        <Route path="/" element={<h1>Home</h1>} />
      </Routes>
    </Router>
  );
}
```